

3008ICT Deep Learning

Week 8. RNNs in Natural Language Processing (NLP)

- Language model (n-gram model)
- Neural models
 - Fixed window models
 - Sequence2sequence
- Applications in NLP

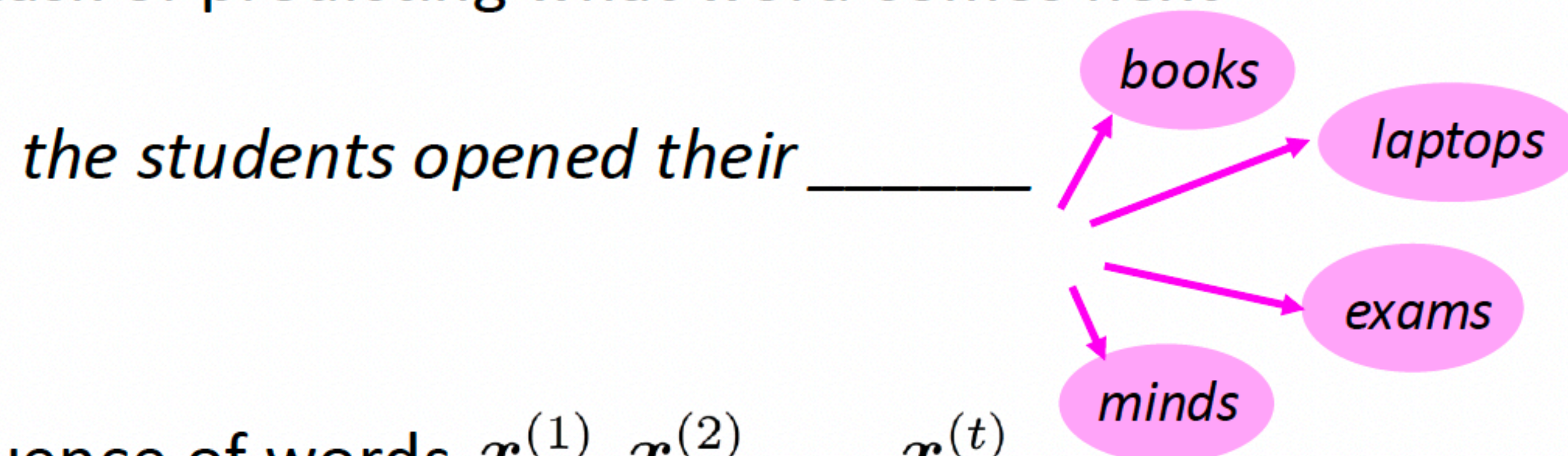
Notices

- Assignment 2 is released (due on 18 May).
- Any questions?

Simple Language Modeling

Language Modeling

- **Language Modeling** is the task of predicting what word comes next



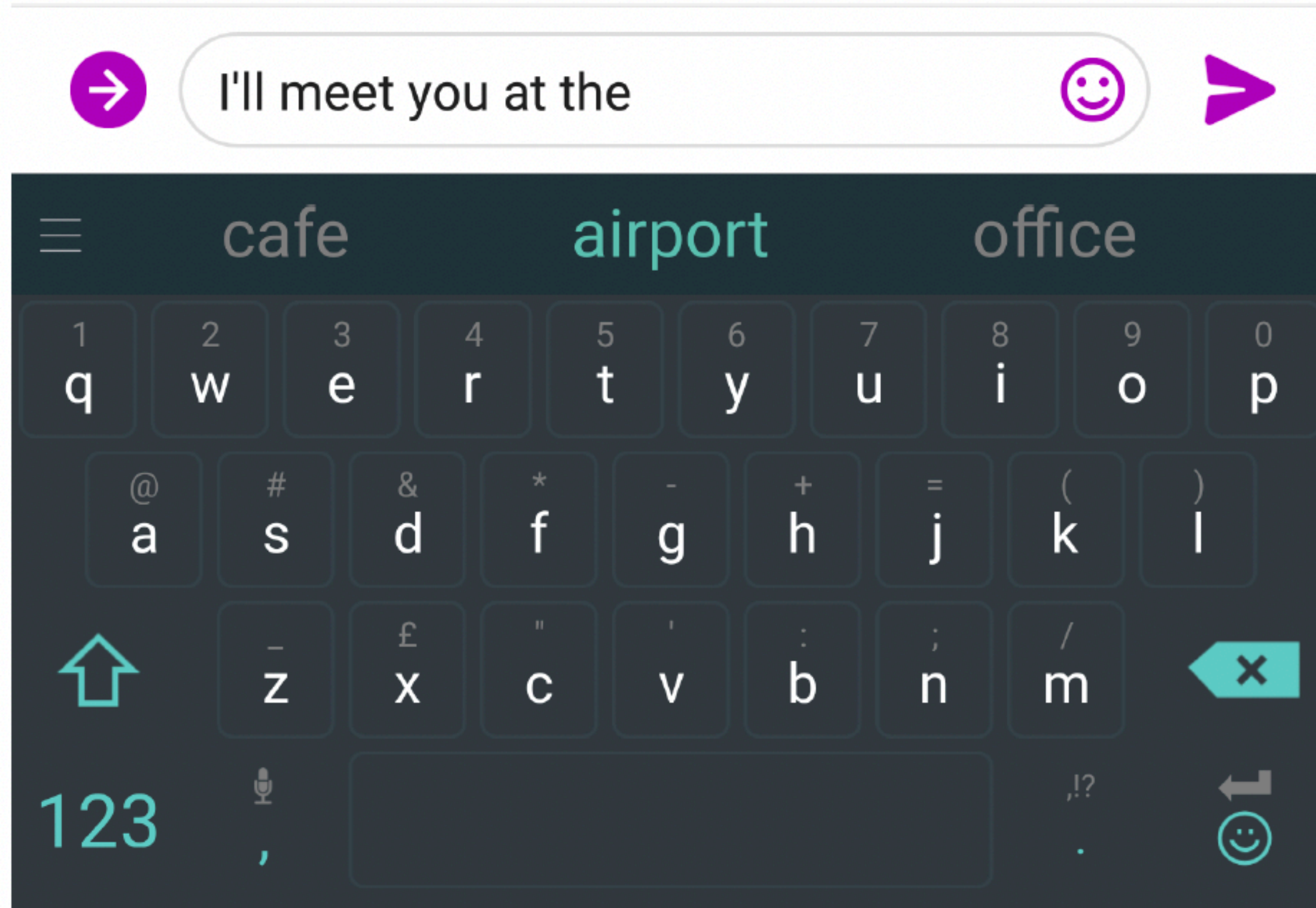
- More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)})$$

where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

- A system that does this is called a **Language Model**

You use Language Models every day!



what is the |

what is the **weather**
what is the **meaning of life**
what is the **dark web**
what is the **xfi**
what is the **doomsday clock**
what is the **weather today**
what is the **keto diet**
what is the **american dream**
what is the **speed of light**
what is the **bill of rights**

Google Search

I'm Feeling Lucky

Why should we care about Language Modeling?

- Language Modeling is a **benchmark task** that helps us **measure our progress** on predicting language use
- Language Modeling is a **subcomponent** of many NLP tasks, especially those involving **generating text** or **estimating the probability of text**:
 - Predictive typing
 - Speech recognition
 - Handwriting recognition
 - Spelling/grammar correction
 - Authorship identification
 - Machine translation
 - Summarization
 - Dialogue
 - etc.
- Everything else in NLP has been rebuilt upon Language Modeling: **ChatGPT is an LM!**

Examples of Language Modeling

The theme park Sea World is located in _____, Queensland. [Trivia]

I put ____ fork down on the table. [syntax]

Alice went to the park. ____ enjoyed the fresh air. [coreference]

I went to the ocean to see the fish, turtles, seals, and _____. [lexical semantics/topic]

Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was _____. [sentiment]

Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the _____. [some reasoning – this is harder]

I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, _____ [some basic arithmetic]

n-gram Language Models

the students opened their _____

- Question: How to learn a Language Model?
- Answer (pre- Deep Learning): learn an n-gram Language Model!
- Definition: An n-gram is a chunk of n consecutive words.
 - unigrams: “the”, “students”, “opened”, “their”
 - bigrams: “the students”, “students opened”, “opened their”
 - trigrams: “the students opened”, “students opened their”
 - four-grams: “the students opened their”
- Idea: Collect statistics about how frequent different n-grams are and use these to predict next word.

n-gram Language Models

- First we make a **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

prob of a n-gram $\rightarrow$

prob of a (n-1)-gram $\rightarrow$

$$\begin{aligned} &= P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}) \\ &= P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}) \end{aligned} \quad (\text{definition of conditional prob})$$

- Question:** How do we get these n -gram and $(n-1)$ -gram probabilities?
- Answer:** By **counting** them in some large corpus of text!

$$\approx \frac{\text{count}(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{\text{count}(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \quad (\text{statistical approximation})$$

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____
discard condition on this

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- “students opened their” occurred 1000 times
 - “students opened their books” occurred 400 times
 - $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$
 - “students opened their exams” occurred 100 times
 - $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$
- Should we have discarded the “proctor” context?

Drawbacks of n-gram models

- Data Sparsity and Zero Probability Problem
- Short-Term Memory
- Lack of Semantic Understanding
- Poor Generalization to New Data

n-gram models are fast and efficient for small-scale applications.
n-grams don't work for large application.
n-grams are now replaced with Transformers

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “*students opened their w*” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$P(w|\text{students opened their}) =$

count(students opened their w)

count(students opened their)

Sparsity Problem 2

Problem: What if “*students opened their*” never occurred in data? Then we can’t calculate probability for *any* w !

(Partial) Solution: Just condition on “*opened their*” instead. This is called *backoff*.

Note: Increasing n makes sparsity problems *worse*. Typically, we can’t have n bigger than 5.

Storage Problems with n-gram Language Models

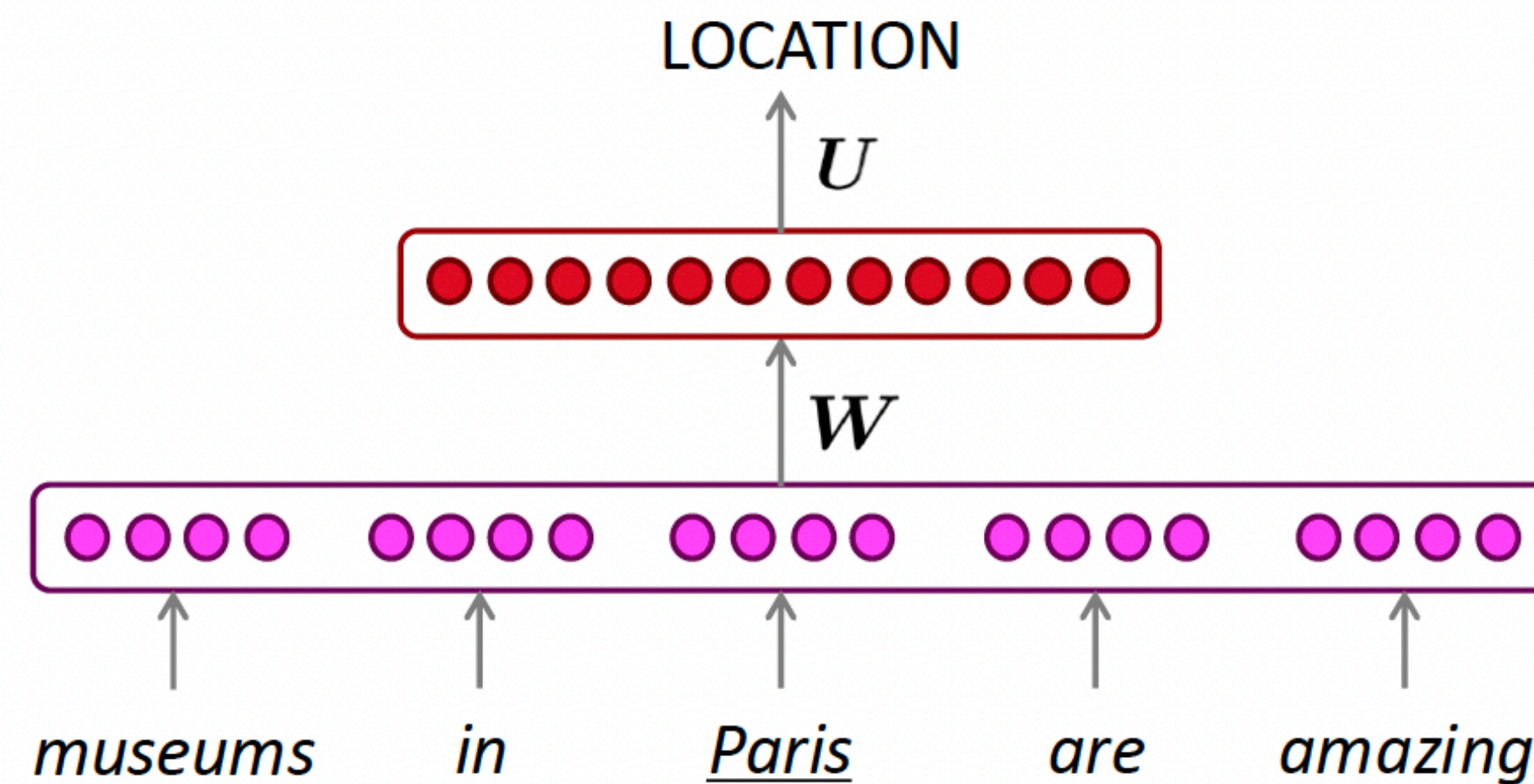
Storage: Need to store count for all n -grams you saw in the corpus.

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Increasing n or increasing corpus increases model size!

How to build a neural language model?

- Recall the Language Modeling task:
 - Input: sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - Output: prob. dist. of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$
- How about a **window-based neural model**?



A fixed-window neural Language Model

~~as the proctor started the clock~~
discard

the students opened their _____
fixed window

A fixed-window neural Language Model

output distribution

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

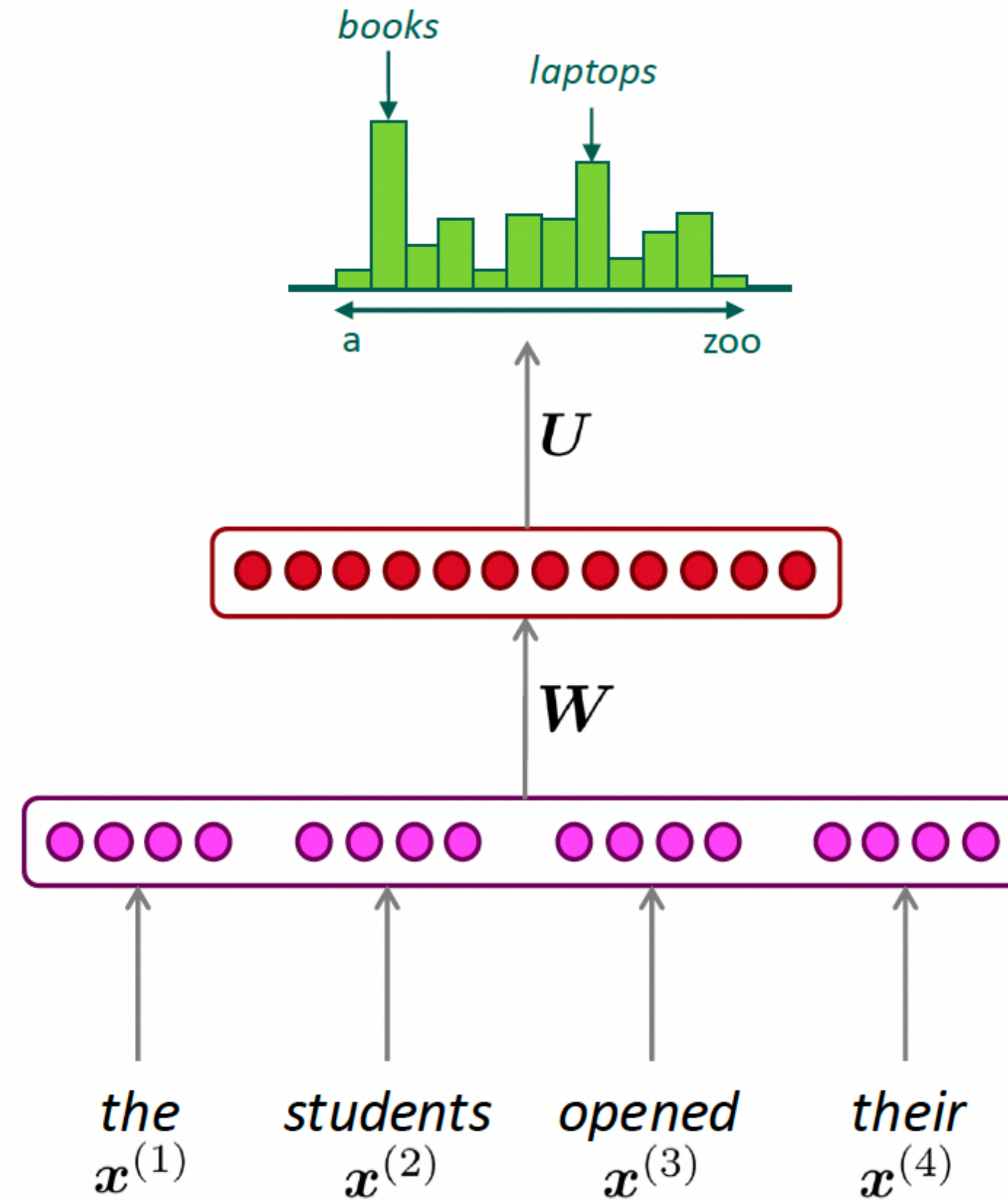
$$\mathbf{h} = f(W\mathbf{e} + \mathbf{b}_1)$$

concatenated word embeddings

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

Improvements over n -gram LM:

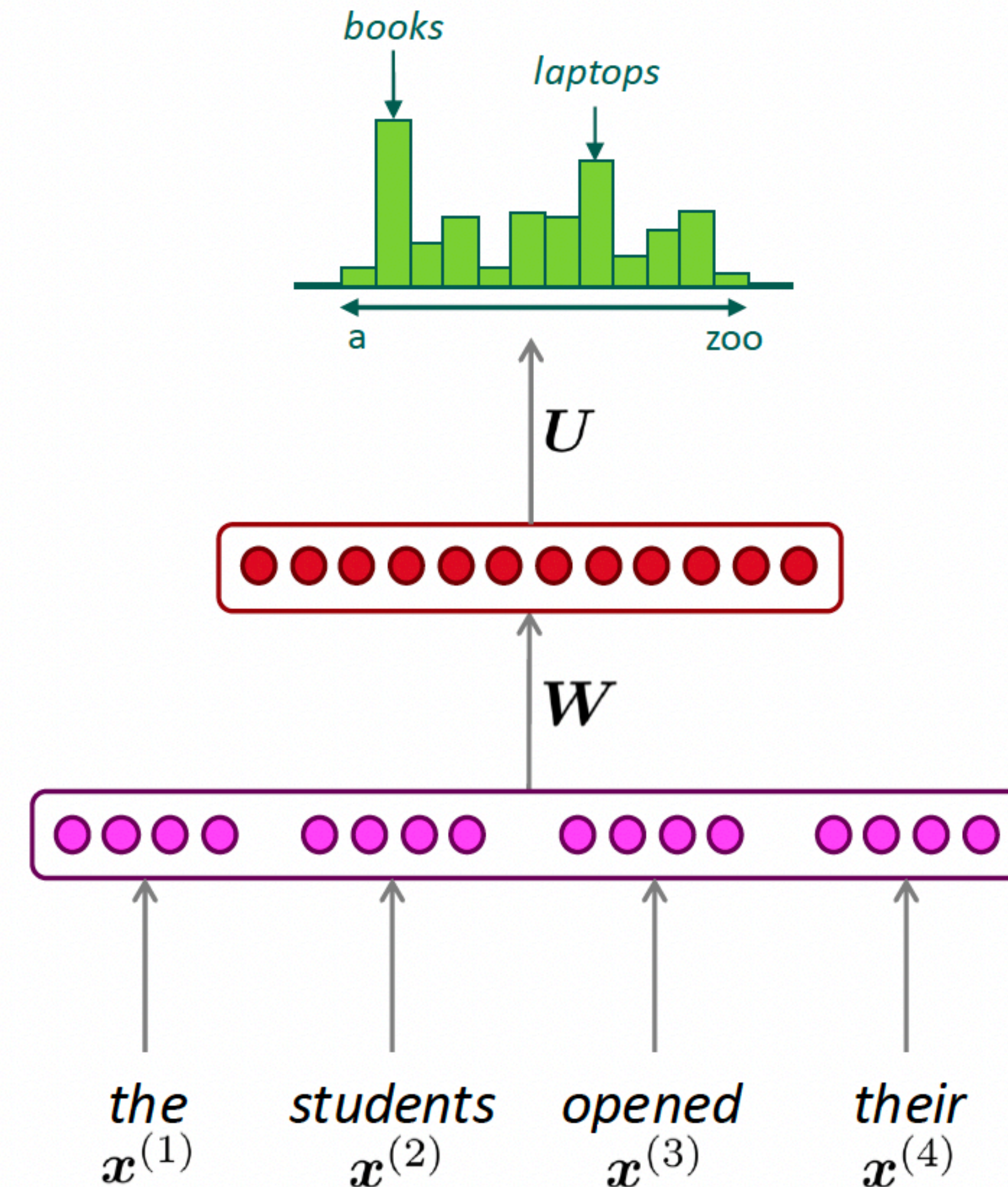
- No sparsity problem
- Don't need to store all observed n -grams

Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .

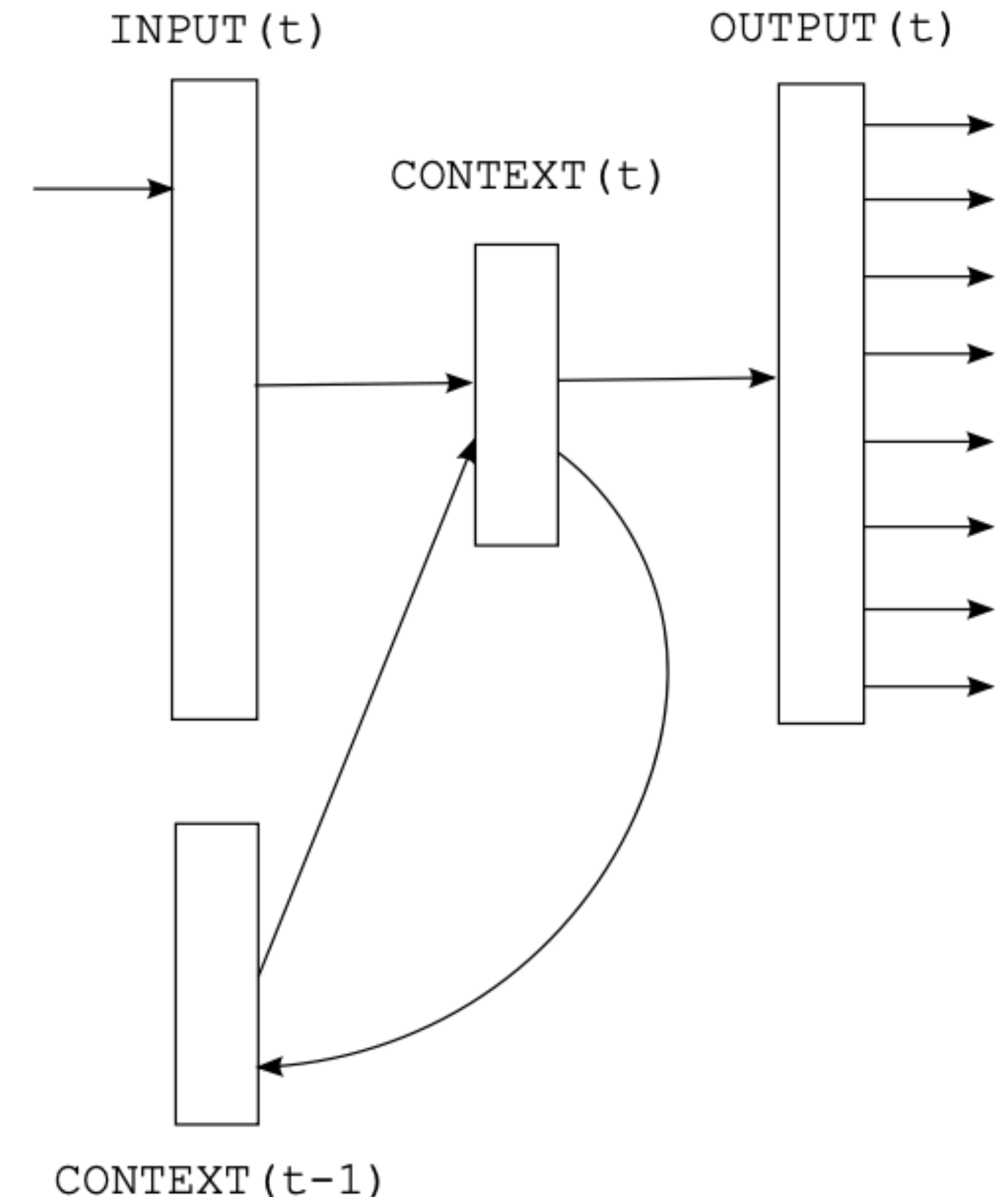
No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*



RNN Solution to Language Modeling

- The Big Idea: Moving from fixed windows to dynamic context.
- Input Layer: Takes in the current word AND the network's memory from the previous step.
- Hidden Layer: The "brain." It mashes the current word and the past memory together to understand the current context.
- Output Layer: Makes a prediction for the next word based on this new understanding.
- The Loop: The new hidden state is instantly copied and becomes the "memory" for the next word.

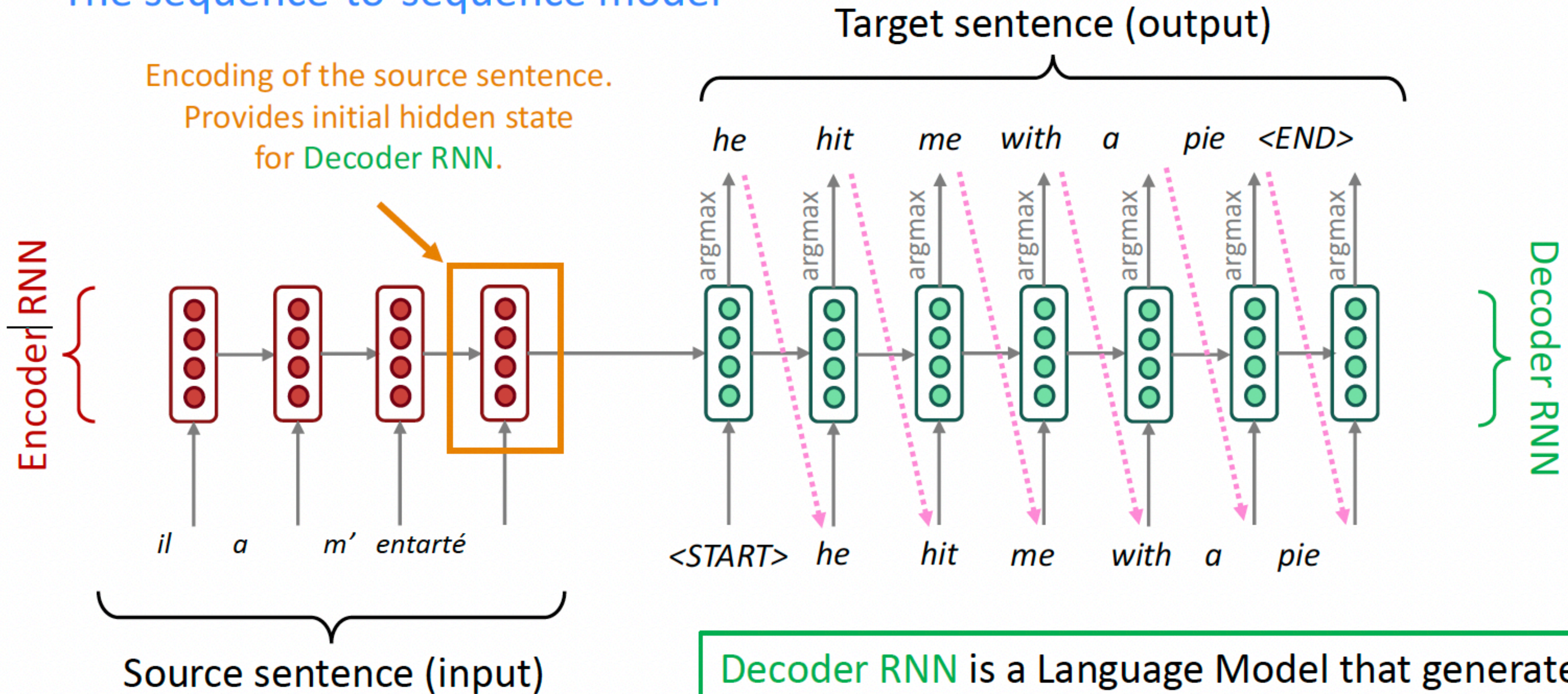


seq2seq for NMT

- Neural Machine Translation (NMT) is a way to do Machine Translation with a single end-to-end neural network
- The neural network architecture is called a sequence-to-sequence model (aka seq2seq) and it involves two RNNs

Neural Machine Translation (NMT)

The sequence-to-sequence model



Encoder RNN produces an **encoding** of the source sentence.

Decoder RNN is a Language Model that generates target sentence, *conditioned on encoding*.

This diagram shows test time behaviour: decoder output is fed in as next step's 's input

Sequence-to-sequence is versatile!

- Sequence-to-sequence is useful for more than just MT
- Many NLP tasks can be phrased as seq2seq:
 - Summarization (long text → short text)
 - Dialogue (previous utterances → next utterance)
 - Parsing (input text → output parse as sequence)
 - Code generation (natural language → Python code)

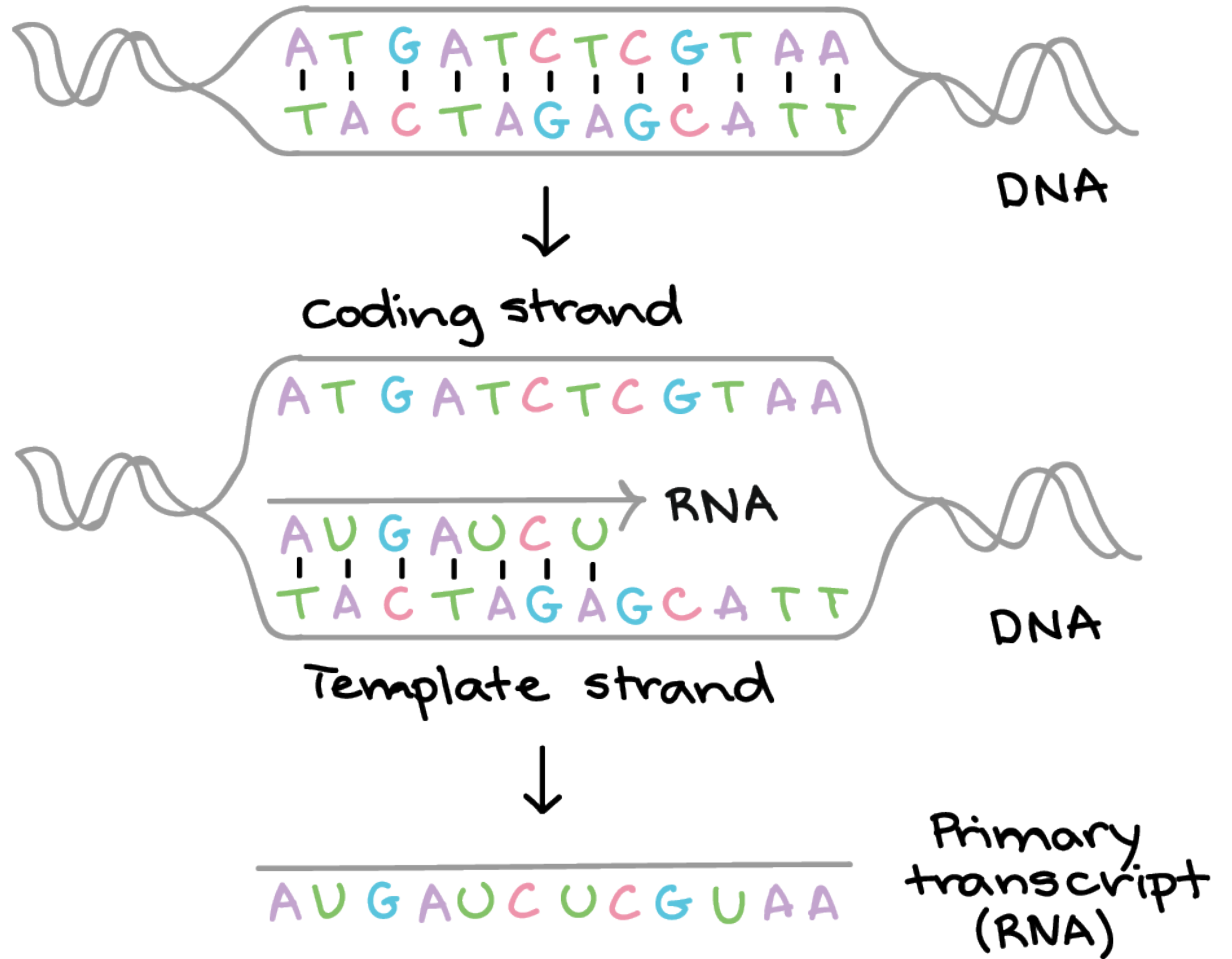
The Base Class for an Encoder

Code: Assignment 2

The Base Class for a Decoder

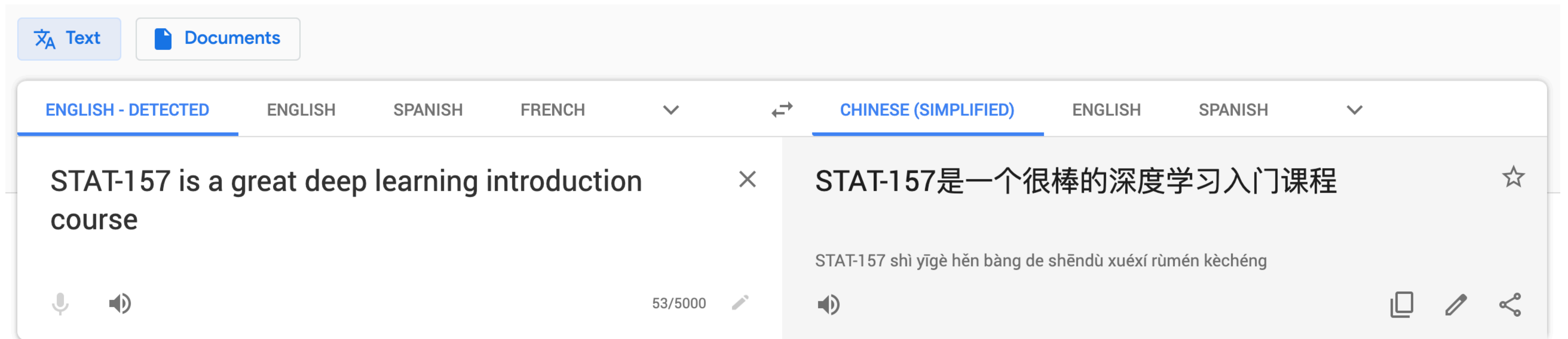
Code: Assignment 2

Seq2seq



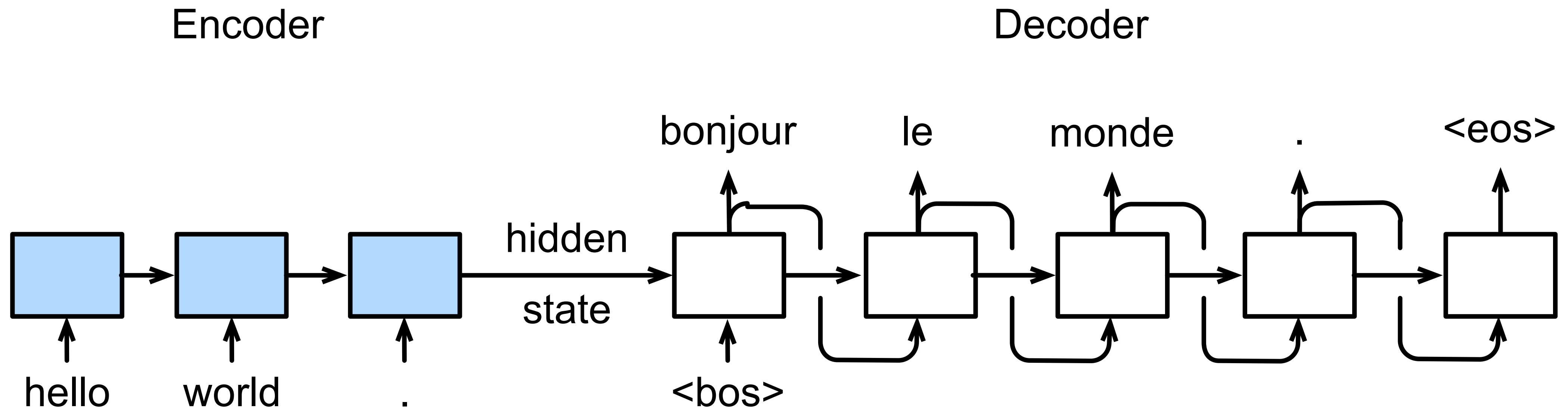
Machine Translation

- Given a sentence in a source language, translate into a target language
- These two sequences may have different lengths



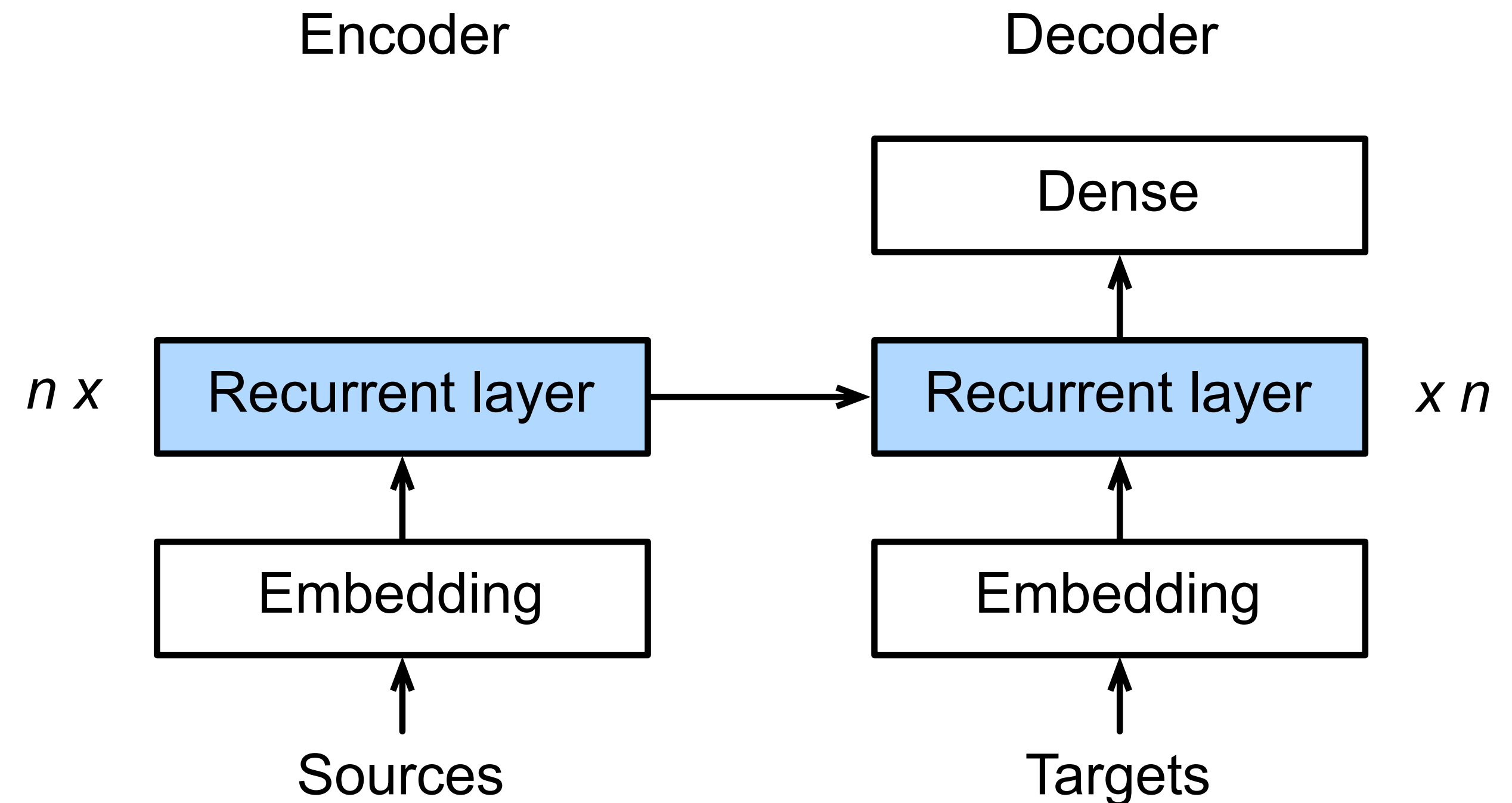
Seq2seq

- The encoder is a RNN to read input sequence
- The decoder uses another RNN to generate output



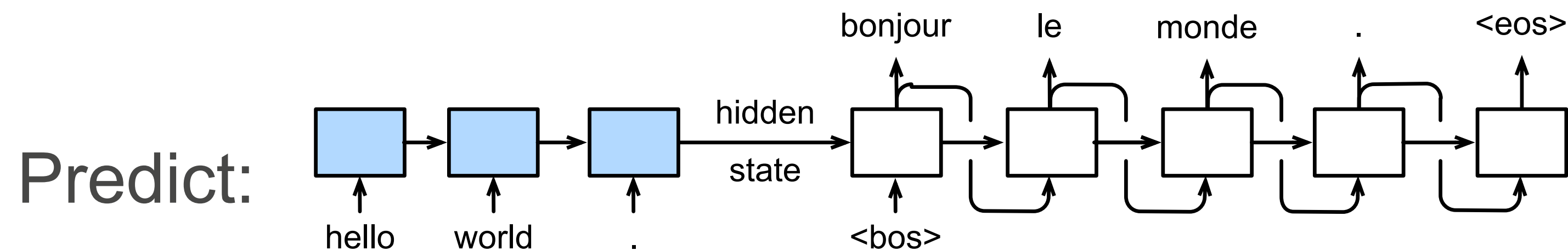
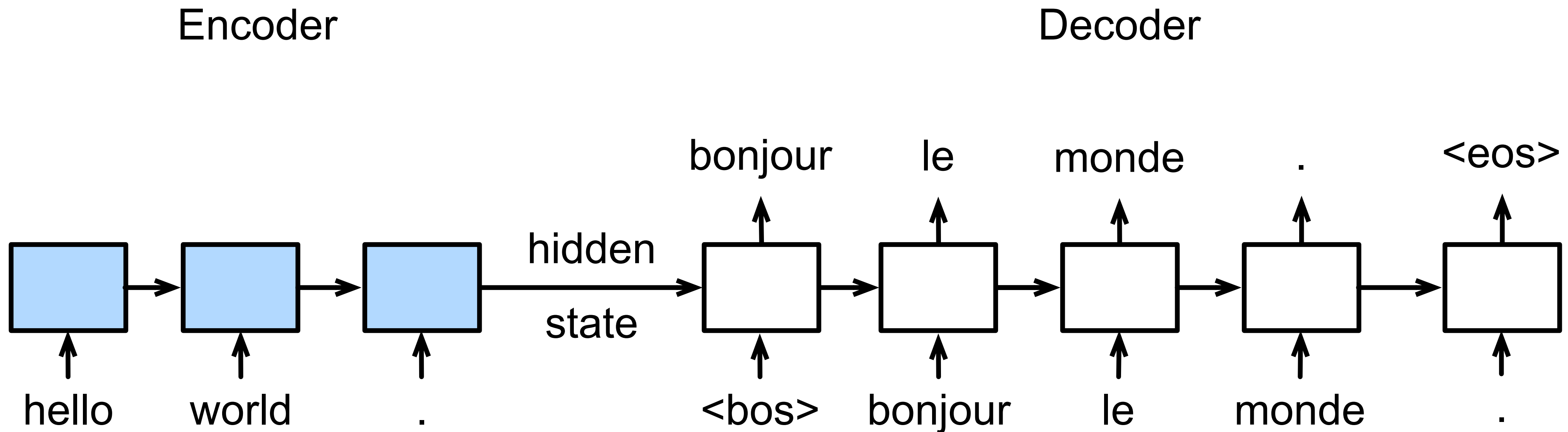
Encoder/Decoder Details

- The encoder is a standard RNN model without the output layer
- The encoder's hidden state in last time step is used as the decoder's initial hidden state



Training

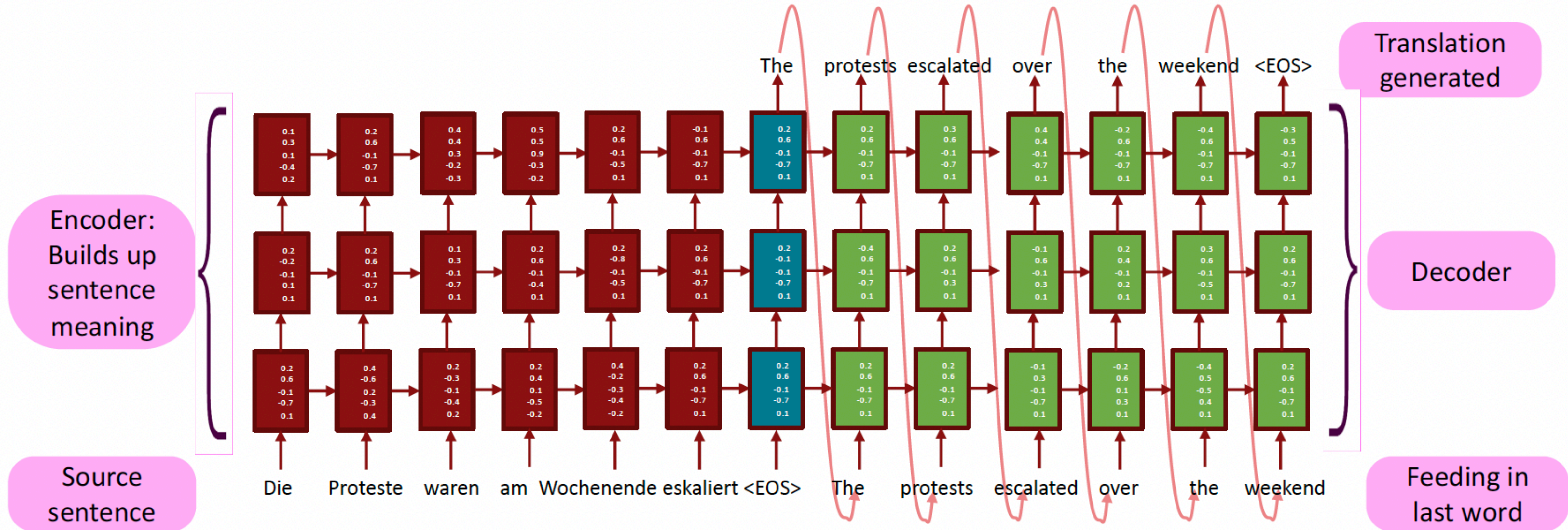
- The decoder is feed with the targeted sentence during training



Multi-layer deep encoder-decoder machine translation net

[Sutskever et al. 2014; Luong et al. 2015]

The hidden states from RNN layer i are the inputs to RNN layer $i+1$



Conditioning =
Bottleneck

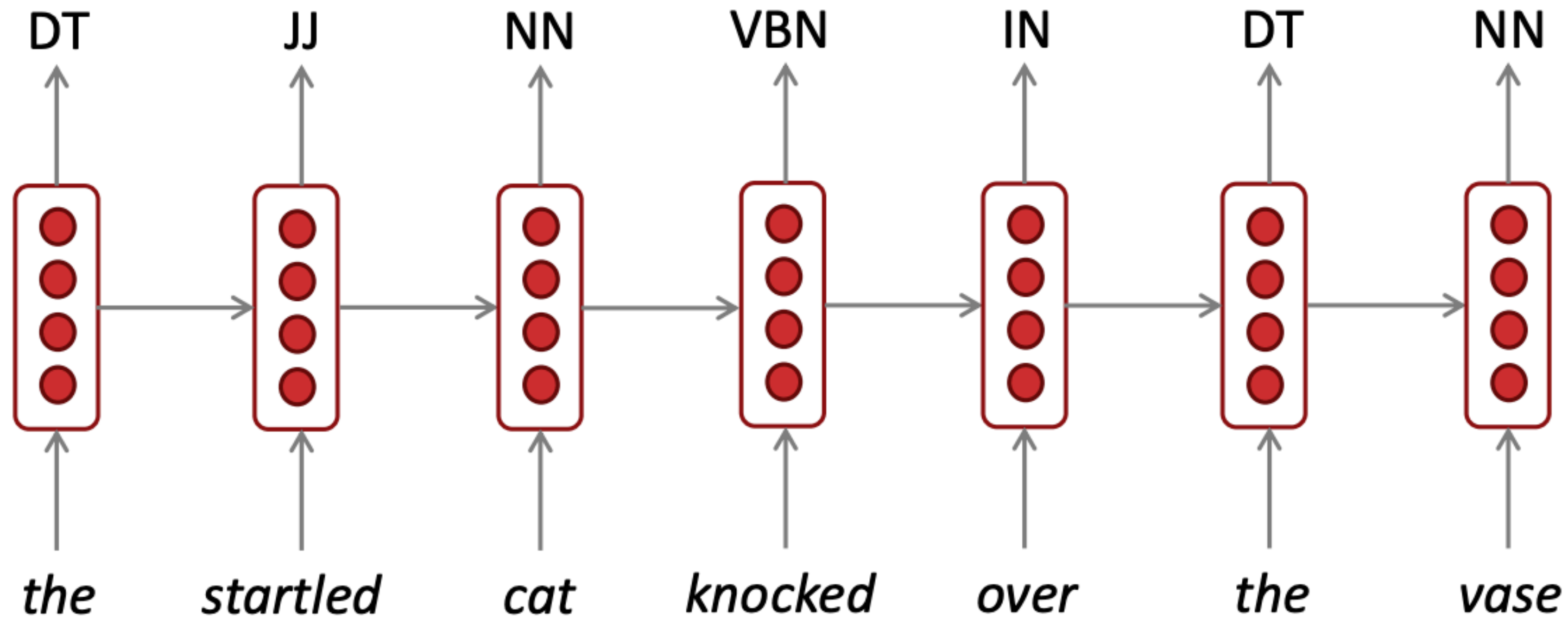
- High-performing RNNs are usually multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For example, for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
 - Often 2 layers is a lot better than 1, and 3 might be a little better than 2
 - Usually, skip-connections/dense-connections are needed to train deeper RNNs (e.g., 8 layers)
- Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.
- You will learn about Transformers later; they have a lot of skipping-like connections

Best of Both Worlds

- The Reality Check: RNNs were great at deep, long-term meaning, but older n-gram models were still faster at catching rigid, immediate grammar.
- The Solution: A Hybrid Model.
- Impact: Combining the two systems led to massive leaps in speech recognition accuracy, proving that neural models were the future of NLP.
- We will talk about more recent technologies later.

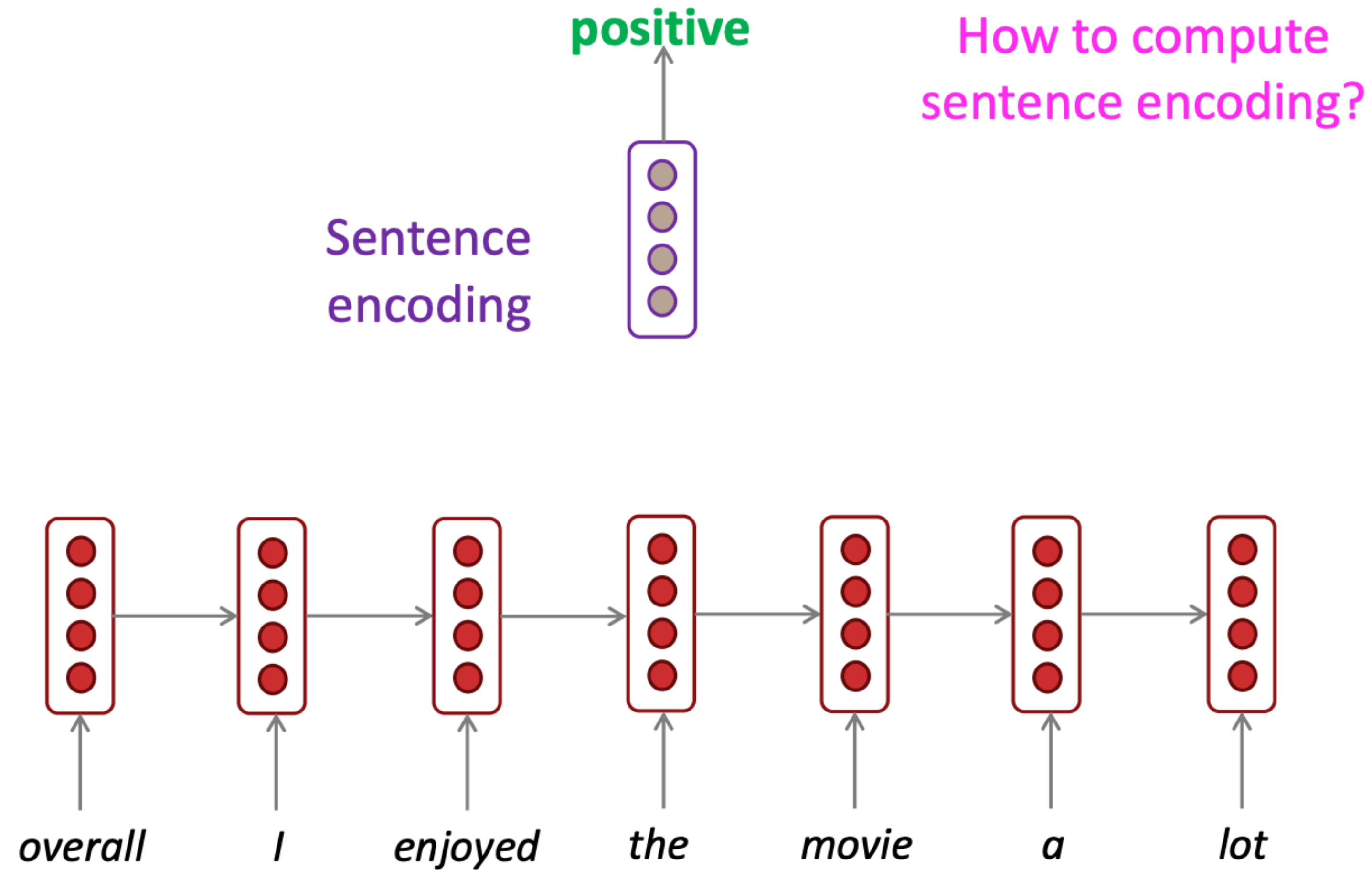
RNNs can be used for tagging

e.g., part-of-speech tagging, named entity recognition



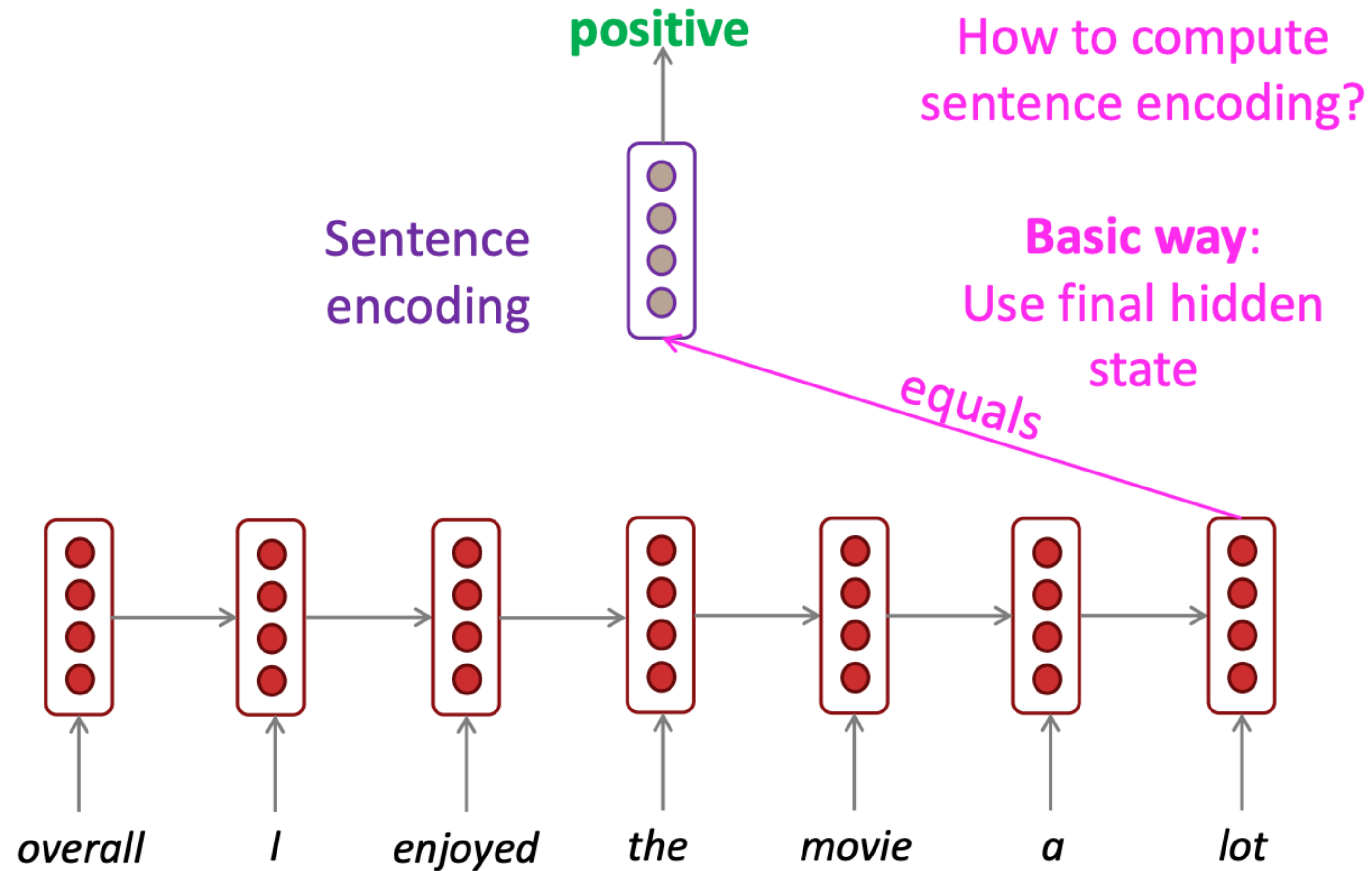
RNNs can be used for sentence classification

e.g., sentiment classification



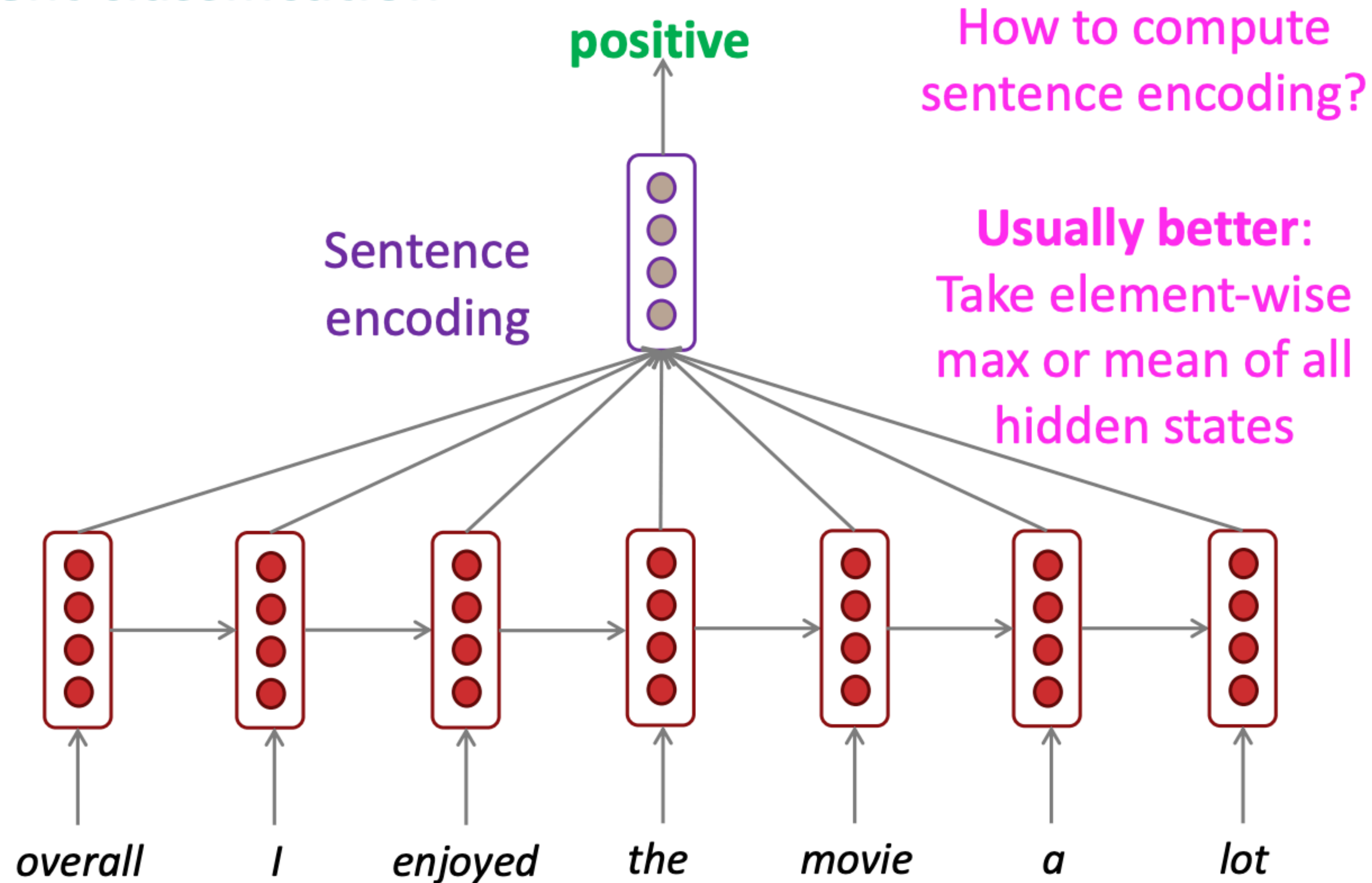
RNNs can be used for sentence classification

e.g., sentiment classification



RNNs can be used for sentence classification

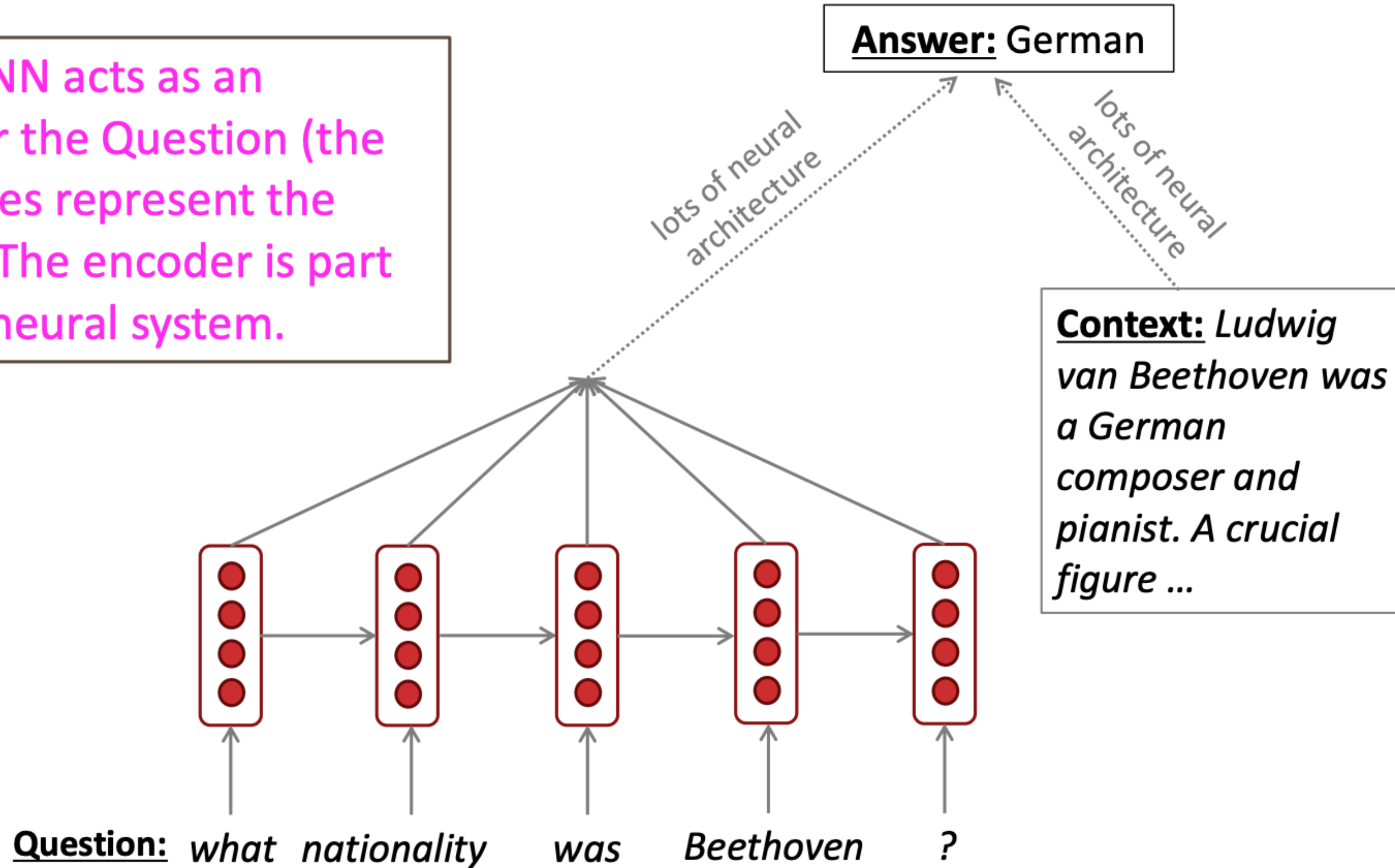
e.g., sentiment classification



RNNs can be used as an encoder module

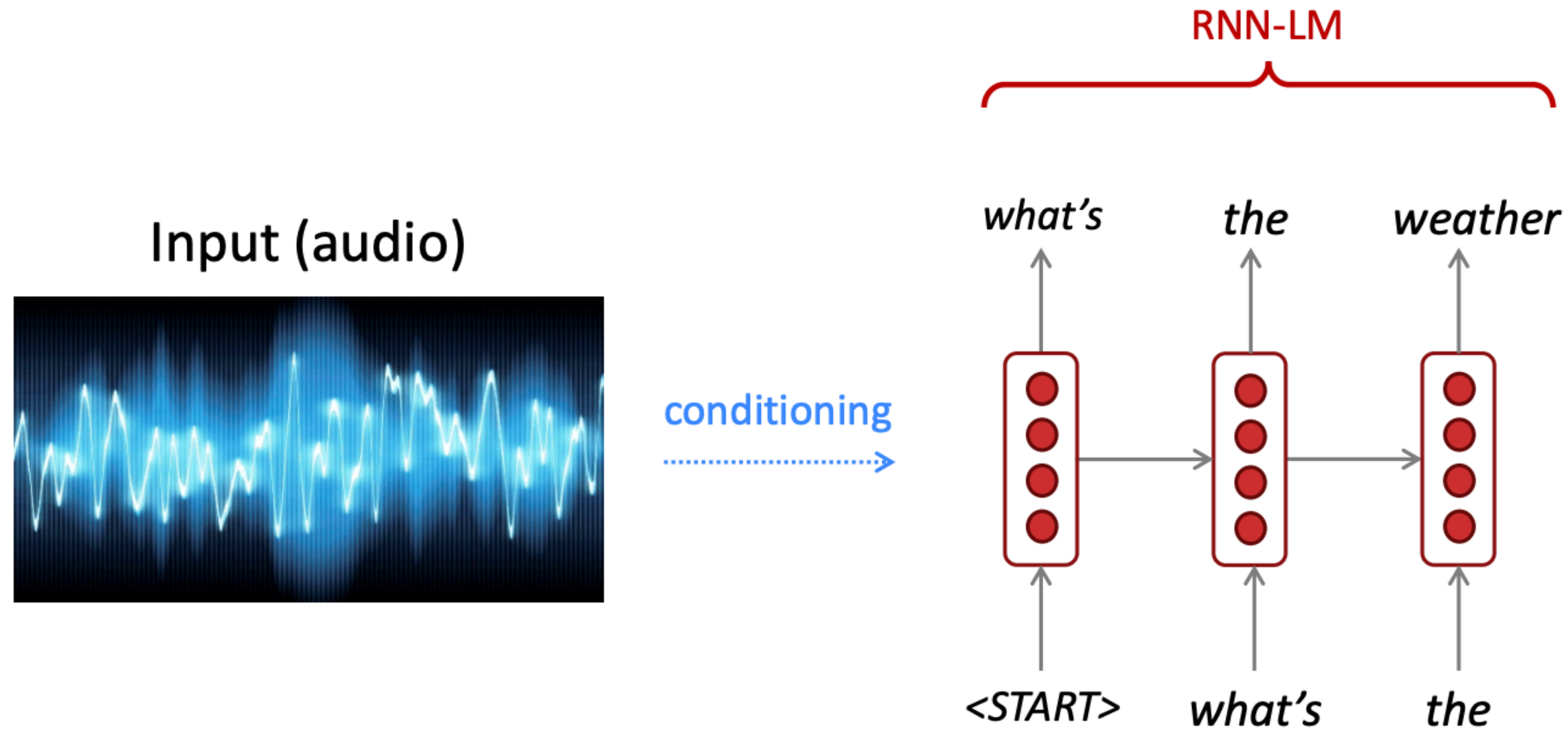
e.g., question answering, machine translation, *many other tasks!*

Here the RNN acts as an **encoder** for the Question (the hidden states represent the Question). The encoder is part of a larger neural system.



RNN-LMs can be used to generate text

e.g., speech recognition, machine translation, summarization



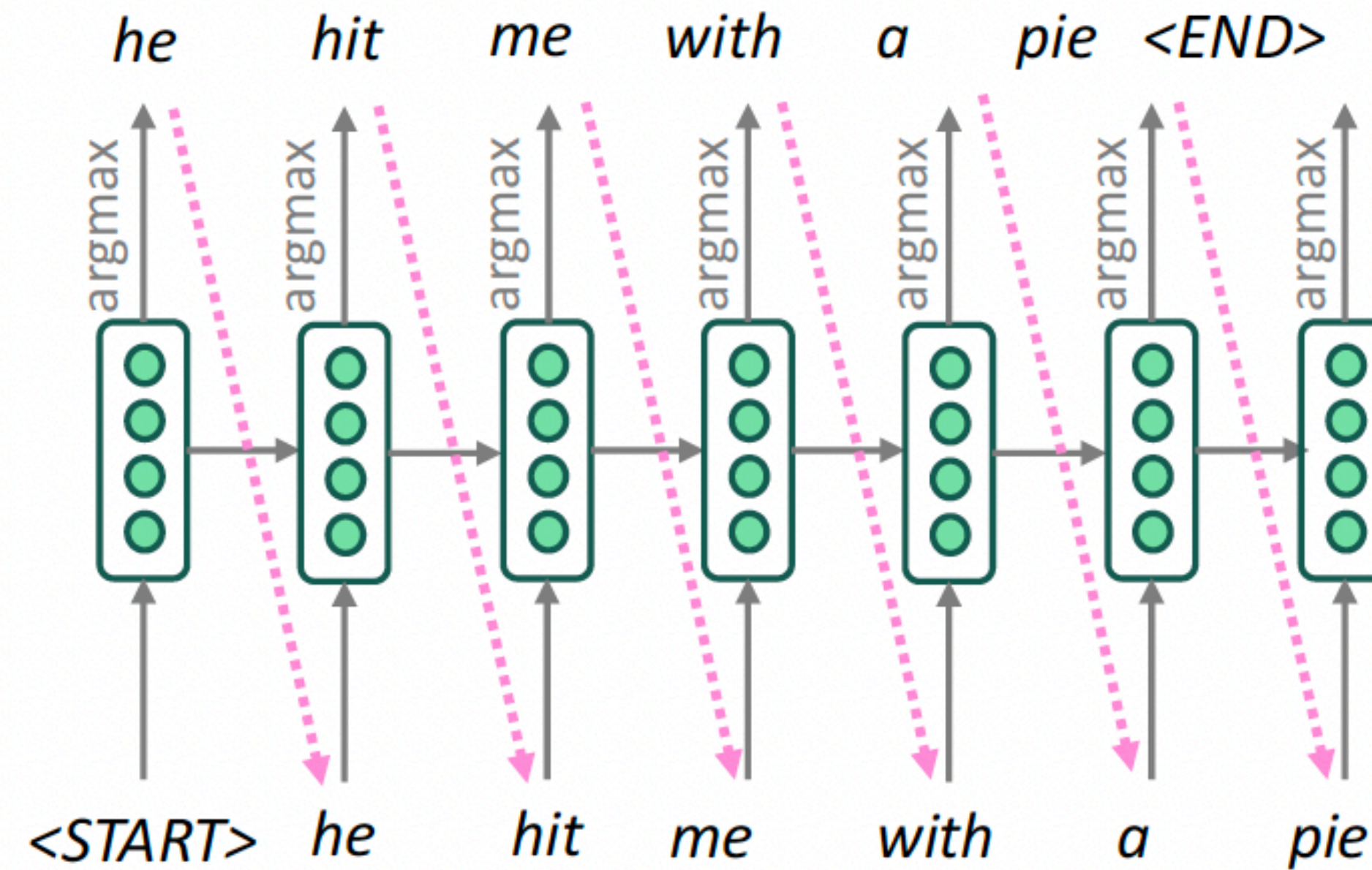
This is an example of a *conditional language model*.
We'll see Machine Translation in much more detail later.

Seq2seq Decoding: Beam Search



Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking argmax on each step of the decoder



- This is **greedy decoding** (take most probable word on each step)
- **Problems with this method?**

Greedy Search

- We used greedy search in the seq2seq model during predicting
- It could be suboptimal

Greedy search:

$$0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$$

Time step	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

A better choice:

$$0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$$

Time step	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

Problems with greedy decoding

- Greedy decoding has no way to undo decisions!
 - Input: *il a m'entarté* (he hit me with a pie)
 - → *he* _____
 - → *he hit* _____
 - → *he hit a* _____ *(whoops! no going back now...)*
- How to fix this?

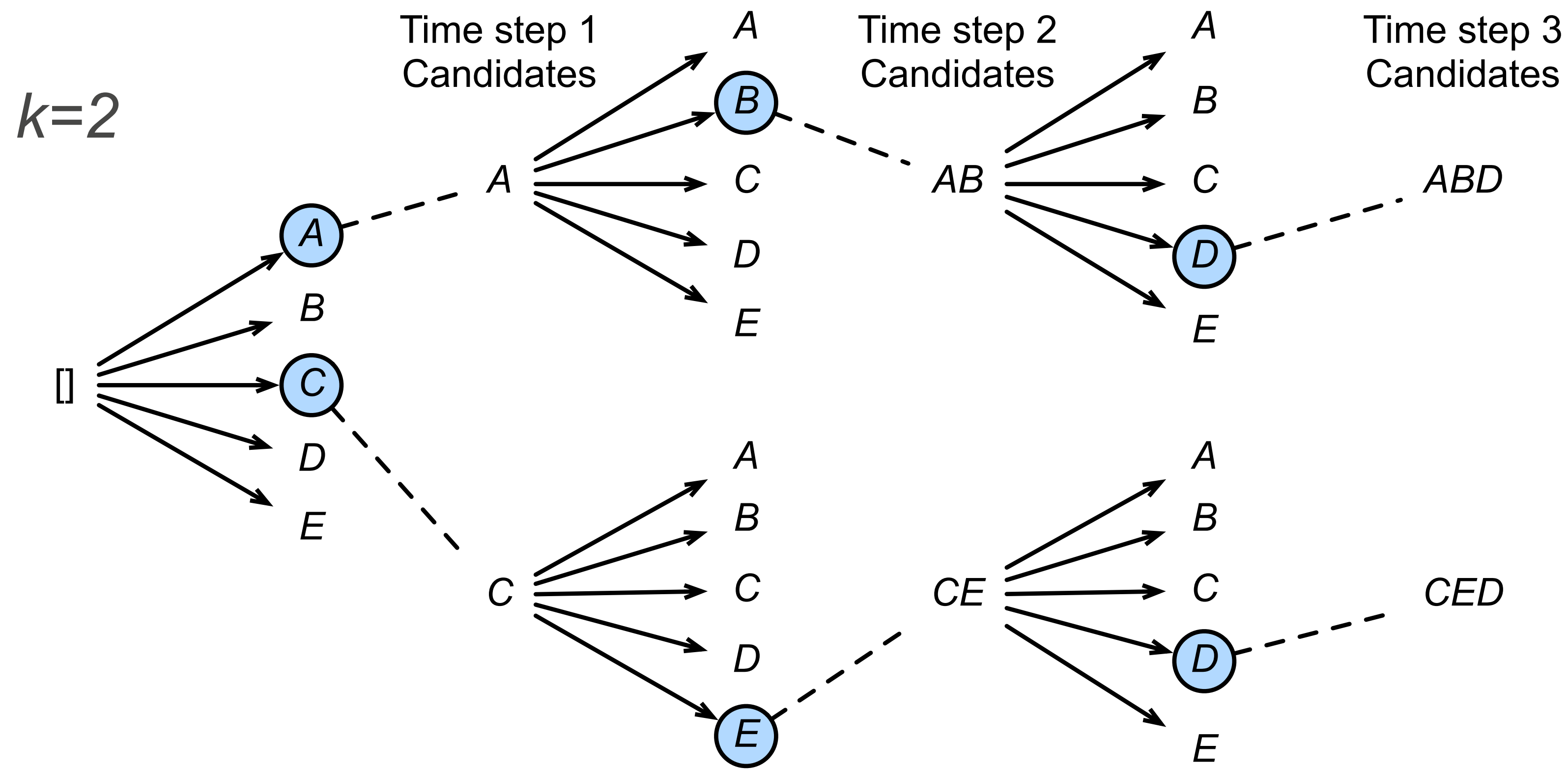
Exhaustive Search

- For every possible sequence, compute its probability and pick the best one
- If output vocabulary size is n , and max sequence length T , then we need to examine n^T sequences
- It's computationally infeasible

$$n = 10000, \quad T = 10 : \quad n^T = 10^{40}$$

Beam Search

- We keep the best k (beam size) candidates for each time
- Examine kn sequences by adding an new item to a candidate, and then keep the top- k ones



Beam search decoding: stopping criterion

- In **greedy decoding**, usually we decode until the model produces an **<END> token**
 - **For example:** *<START> he hit me with a pie <END>*
- In **beam search decoding**, different hypotheses may produce <END> tokens on **different timesteps**
 - When a hypothesis produces <END>, that hypothesis is **complete**.
 - **Place it aside** and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach timestep T (where T is some pre-defined cutoff), or
 - We have at least n completed hypotheses (where n is pre-defined cutoff)

Beam search decoding: finishing up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- **Problem with this:** longer hypotheses have lower scores
- **Fix:** Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Summary

- Language Model: A system that predicts the next word
- RNNs are a great way to build a LM.
- Embeddings are Lossy: They optimize for similarity, not reconstruction. Always keep your source text.
- Tokenization Matters: It is the foundation of consistency.
- Balance your DB Choice: Weigh Accuracy vs. Speed, and Cost vs. Performance.
- A language model has many applications.